

A Reproduction of SkillOS Under Contemporaneous Controls: The Curator Lift Does Not Survive a Same-Epoch Baseline

Omer Karisman
ok@inference.sh

The experiments, implementation, analysis, and first draft of this paper were produced by an LLM agent (Claude, Anthropic) under the author’s direction. This is a material fact about the study’s reliability, not an acknowledgement: see Section 4.5, Section 6.3, and Appendix A.

August 19, 2026

Abstract

SkillOS trains a curator model with GRPO to maintain a markdown skill repository for a frozen executor, and reports a 13.3 percentage point improvement in ALFWorld task success. We attempted to reproduce this over roughly two months on 8xH100, across seven complete 60-step training runs, two independent RL frameworks (TRL with ZeRO-3 and verl-agent/GiGPO with FSDP), three seeds, two executor scales, and approximately one hundred evaluation arms of 140 paired games each.

For most of that period we believed we had reproduced the effect: every checkpoint sweep contained a significant-looking peak in the +9 to +14pp band, consistently across seeds and frameworks. Those peaks were artifacts of a control measured once and reused. Re-measuring the no-memory baseline in the same week as the arms moved it from 33.6% to 39.8% (four replicates, spread 2.1pp), a shift larger than most effects claimed in this literature. Re-paired against a contemporaneous control, no ALFWorld checkpoint trained on ALFWorld shows a significant lift on the training executor, and our strongest previous result (+13.6pp, $p=0.0026$) becomes +3.6pp, $p=0.47$.

We report four surviving results. First, a curator trained on **mathematics** lifts ALFWorld success by +9.0pp on a held-out split ($p=0.073$), while curators trained on ALFWorld itself do not; cross-domain transfer is the only positive signal we found. Second, substituting Gemini 2.5 Pro for the trained curator, at 84 times the cost per call, yields no improvement over writing no notes at all. Third, supplying retrieved skills increases this executor’s rate of unparseable actions from 2.1% to 7.0-9.1%, a concrete mechanism by which memory can fail to help a small model. Fourth, and independent of SkillOS: the standard 140-game ALFWorld protocol has 80% power to detect a 13.0pp effect and no less, meaning the field’s headline effect sizes sit exactly at the resolution limit of the instrument used to measure them.

We make no claim that the original result is wrong. We report that a faithful implementation, run seven times, did not yield it under contemporaneous controls, and that our absolute baseline remains 8pp below the original’s after eliminating six candidate causes. All checkpoints, evaluation records, and analysis code are released so that every test here can be recomputed.

This study was conducted by an LLM agent under human direction. Both of its consequential errors, disclosed in full, produced *more* statistically significant findings rather than fewer, because

a systematically degraded arm is reliably degraded and reliability is what a significance test rewards. We discuss the implications for autonomous experimentation.

1. Introduction

An agent that writes notes to itself and gets better at its job is an appealing idea. It promises improvement without touching the weights of the model doing the work, which means it should apply to any frozen or closed model you can call over an API.

SkilIOS (Ouyang et al., 2026) makes this concrete. A *curator* model, trained with GRPO, maintains a repository of markdown skills. A separate *executor* model, frozen throughout, reads the retrieved skills and acts. The reported result on ALFWorld is a 13.3 percentage point improvement in task success rate over the same executor with no skills, and 61.2% absolute with a 32B executor.

We set out to reproduce this. The motivation was practical rather than critical: if a small trained curator can lift a frozen agent by 13pp, that is a cheap and deployable technique, and we wanted it working before building on it.

This paper reports what happened over roughly two months of 8xH100 time and seven complete 60-step training runs. The short version is that we could not obtain the effect, and that the most useful thing we learned was about measurement rather than about skill repositories.

1.1 What we did

We implemented the method in two independent reinforcement learning stacks: TRL with DeepSpeed ZeRO-3, and verl-agent (GiGPO) with FSDP. We trained a Qwen3-8B curator against a frozen Qwen3-8B executor on ALFWorld, with a Qwen3-32B judge supplying the consistency reward term, following the paper’s hyperparameters. We swept checkpoints every five steps and evaluated each one on 140 held-out ALFWorld games, paired by game file and tested with McNemar.

Along the way we ran the paper’s own ablations (task-type distribution, within-group curriculum ordering), a LoRA-versus-full-fine-tuning comparison, a decode-parameter sweep, a cross-executor-scale transfer study, a cross-domain study training the curator on mathematics and testing on ALFWorld, and an additional arm the paper motivates but does not run: substituting a frontier model, Gemini 2.5 Pro, for the trained curator.

1.2 What we found

For most of those two months we believed we were reproducing the paper. Every sweep contained a checkpoint with a significant-looking lift, in the +9 to +14pp band that brackets the reported +13.3pp. Three seeds and two frameworks agreed.

They agreed because they were all being compared against the same control number, measured once, ten weeks before the arms that were being compared to it.

When we re-measured that control in the same week as the arms, it moved from 33.6% to 39.8% (four replicates, spread 2.1pp). The old figure sits outside the replicate spread, so it is not sampling noise. Re-paired against a contemporaneous baseline, the lifts disappear: our best seed goes from +13.6pp at $p=0.0026$ to +3.6pp at $p=0.47$, and eight of its twelve checkpoints are negative.

Our principal findings are therefore:

1. **The curator lift does not reproduce under a same-epoch control.** We make no claim that the original result is wrong. We report that a faithful implementation, run seven times, did not yield it once the control was measured correctly.

2. **Reusing a control measured against a hosted model API is a significance factory.** The drift we measured (5.7pp) is larger than most effects being claimed in this literature. This is a methodological result independent of SkillOS.
3. **A frontier curator does no better.** Gemini 2.5 Pro, at 84 times the cost per call, scores 1.4pp below writing no notes at all ($p=0.86$). The directional part of the paper’s claim, that a small trained curator is competitive with a much larger one at this task, survives. The part that would make it exciting does not, because neither of them beats the baseline.
4. **Adding skills makes a small executor produce more unparseable actions**, from 2.1% with no memory to 7.0-9.1% with curation. This is a concrete mechanism for why memory can fail to help, or actively hurt, a small model.
5. **Six candidate explanations for the null are falsified**, each with a full training run or a full sweep behind it, including both halves of the paper’s own grouping ablation.

1.3 What this paper is not

It is not a refutation. Reproduction failures have many causes, and the most likely one is always that the reproducers got something wrong. We list what we know we did differently in Section 4.4 and what we suspect but cannot rule out in Section 6. Our absolute baseline sits 8pp below the paper’s and we could not close that gap after eliminating six causes, which is itself evidence that something in our setup differs from theirs in a way we have not identified.

What we can offer is the full apparatus: seven trained models, roughly one hundred evaluation arms of 140 paired games, the training and evaluation code, and every JSONL needed to recompute every test in this paper. If the effect is there and we missed it, the artifacts should make it findable.

2. Background and related work

2.1 Externalised memory for frozen agents

A large body of work gives an agent a place to put what it learns, so that the agent improves without gradient updates. Reflexion [Shinn et al. \[2023\]](#) writes verbal self-critiques into the next attempt’s context. Voyager [Wang et al. \[2023\]](#) accumulates an executable skill library in Minecraft. ExpeL [Zhao et al. \[2024\]](#) distils cross-task insights from prior trials. Generative Agents [Park et al. \[2023\]](#) maintain a reflective memory stream, and MemGPT [Packer et al. \[2023\]](#) treats context as a paged resource. Agent Workflow Memory [Wang et al. \[2024\]](#) induces reusable workflows from experience.

By 2026 this had become a dense area, with skill libraries specifically framed as procedural memory and multiple systems for self-evolving them [Shen et al. \[2026\]](#), [Yang et al. \[2026\]](#), [Ni et al. \[2026\]](#), [Huang et al. \[2026\]](#) and at least one survey of the design space [Du \[2026\]](#). A systematic study of model-generated agent skills [Huang et al. \[2026\]](#) is the closest in spirit to what we do here.

Two properties of this literature matter for our purposes. First, the memory is usually produced by prompting rather than trained, so its quality is bounded by the model writing it. Second, evaluation is usually a single number on a small fixed benchmark, with no confidence interval and no correction for the number of configurations tried. Section 5.10 argues that the second property is a serious problem at the effect sizes involved.

2.2 Training the memory writer

SkillOS [Ouyang et al. \[2026\]](#) is a direct response to the first property. The component that writes skills is a separate policy trained with GRPO [Shao et al. \[2024\]](#), rewarded by the downstream success of a frozen executor that consumes the skills. This is an appealing design: it decouples the model that learns from the model that acts, so the acting model can be frozen, closed, or arbitrarily large.

The reported results are strong. On ALFWorld [Shridhar et al. \[2021\]](#) a trained 8B curator improves a frozen executor by 13.3 percentage points, reaches 61.2% absolute with a 32B executor, and is reported to outperform a frontier-model curator at the same job. The paper also reports cross-domain transfer, with a curator trained on mathematics improving ALFWorld performance.

If those results hold, the technique is unusually practical. That is why we tried to reproduce it.

2.3 Reinforcement learning for LLM agents

GRPO [Shao et al. \[2024\]](#) removed the value network from PPO-style RLHF and became the default for reasoning training [DeepSeek-AI \[2025\]](#). Applying it to multi-step agents raises credit assignment problems that episode-level advantages handle poorly; GiGPO [Feng et al. \[2025\]](#) addresses this with a two-level grouping scheme and reports large gains over GRPO on ALFWorld and WebShop [Yao et al. \[2022\]](#). We use both a GRPO implementation (TRL [von Werra et al. \[2020\]](#) with ZeRO-3 [Rajbhandari et al. \[2020\]](#)) and a GiGPO implementation (verl [Sheng et al. \[2024\]](#)), partly to bound framework effects on our conclusions.

2.4 Reproducibility and statistical practice

Our findings are as much about measurement as about skill repositories, so we lean on an older literature.

Deep RL results were shown to be highly sensitive to seeds, implementation details, and reporting choices [Henderson et al. \[2018\]](#), and to require more seeds than are typically run [Colas et al. \[2018\]](#). [Agarwal et al. \[2021\]](#) showed that point estimates on small benchmark suites routinely support conclusions the data cannot bear, and proposed interval-based reporting. [Bouthillier et al. \[2021\]](#) quantified how much benchmark variance comes from sources other than the treatment.

In NLP specifically, [Card et al. \[2020\]](#) performed the analysis that most directly anticipates our Section 5.10: they computed statistical power for common benchmarks and found that most attempted comparisons to state of the art are underpowered, with typical test sets unable to resolve the differences being claimed. [Dror et al. \[2018\]](#) catalogue the significance-testing practices this requires. We apply McNemar’s test [McNemar \[1947\]](#) with Holm [Holm \[1979\]](#) and Benjamini-Hochberg [Benjamini and Hochberg \[1995\]](#) corrections, and report bootstrap intervals throughout, following this line of argument rather than inventing anything.

One thread is newer and specific to the present moment. Models served over hosted APIs are not stationary instruments; [Chen et al. \[2023\]](#) documented measurable behavioural change in a commercial model over a few months. Agent evaluations now routinely place a hosted model inside the measurement loop, often as the frozen component whose behaviour is assumed constant. Section 5.1 reports what this cost us: a control measured against a hosted executor moved 5.7 percentage points in ten weeks, which is larger than most effects claimed in Section 2.1’s literature, and reusing it silently converted endpoint drift into apparent treatment effect across seven training runs.

To our knowledge no prior work quantifies hosted-endpoint drift as a confound *inside an agent evaluation*, and we suggest that contemporaneous control measurement should be a stated requirement for any benchmark that calls a hosted model.

4. Methodology

4.1 The method under test

SkillOS trains a curator policy to maintain a repository of markdown skills that a frozen executor retrieves at inference time. We implemented Algorithm 1 as described in the paper.

For each training prompt the curator observes a task and the current repository, then emits repository operations (add, edit, delete) as function calls. The edited repository is then used by the frozen executor on a sequence of probe tasks, and the executor’s success rate becomes the primary reward term.

The composite reward is the paper’s Equation 1:

$$\mathbf{r} = \mathbf{r_task} + \lambda_f \cdot \mathbf{r_fc} + \lambda_u \cdot \mathbf{r_cnt} + \lambda_c \cdot \mathbf{r_comp}$$

$$\lambda_f = 1.0, \quad \lambda_u = 0.1, \quad \lambda_c = 0.05$$

where $\mathbf{r_task}$ is mean executor success over probe positions $2..|G|$, $\mathbf{r_fc}$ scores function-call validity, $\mathbf{r_cnt}$ is a judged consistency score from a Qwen3-32B judge, and $\mathbf{r_comp}$ scores repository compactness.

4.2 Models, hyperparameters, and hardware

Curator	Qwen3-8B, trained. LoRA r=32 and full fine-tuning
Executor	Qwen3-8B, frozen. Also evaluated with Qwen3-32B
Judge ($\mathbf{r_cnt}$)	Qwen3-32B
Algorithm	GRPO, 60 steps, group size N=8, data grouping size $ G =10$
Effective batch	32 prompts per optimizer update (paper Table 4)
Learning rate	1.0e-6 for full fine-tuning; 1.0e-5 for LoRA
KL coefficient	$\beta = 0.001$
Frameworks	TRL 1.4 + DeepSpeed ZeRO-3 + vLLM colocate; verl-agent (GiGPO) + FSDP
Hardware	8xH100 local for the curator; hosted API for executor and judge inference

The paper used 16 H100s. We used 8 locally and moved executor and judge inference to a hosted endpoint, which we treat as equivalent in capability but not in wall clock.

4.3 Evaluation protocol

Every ALFWorld arm is 140 games from the `valid_seen` split, run under streaming curation: the curator updates the repository between games, so the arm measures the curator as it would actually be deployed. A second protocol, 134 games from `valid_unseen`, is used where noted as a true held-out test for checkpoints selected on `valid_seen`.

Arms are compared **paired by game file** and tested with McNemar’s test on the discordant pairs. Pairing matters more than it usually does here: ALFWorld game difficulty varies enormously by task type, so an unpaired comparison of two 140 game runs is dominated by which games each happened to draw.

Reasoning arms are AIME24 (30 problems), AIME25 (30), and GPQA-Diamond (198). Per the dataset’s access terms we report aggregate accuracies only; no problem text, options, gold answers, or model responses appear in this paper or in the released artifacts.

4.3.1 Contemporaneous baselines

Every arm reported in Section 5 is paired against a no-memory control measured **in the same week, on the same games, under the same harness build**. This is not a routine detail. It is the correction that changed our conclusions, and Section 5.1 reports what happened when we did not do it.

We recommend the practice generally. Executors served over a hosted API are not fixed instruments. A control measured against one is a measurement of that endpoint on that day, and reusing it later silently converts endpoint drift into apparent treatment effect.

4.4 Known deviations from the original

We list these up front because a reproduction failure has to be read against them.

1. **Framework.** The paper’s stack is not ours. TRL with ZeRO-3 and verl-agent with FSDP differ from each other and from the original in advantage normalisation, sequence packing, and optimiser sharding. We ran both partly to bound this.
2. **LoRA in some runs.** Three runs use LoRA $r=32$ with a 10x learning rate rather than full fine-tuning. Full fine-tuning runs are also reported and behave the same way.
3. **Effective batch in the verl run.** The verl run used an effective batch of 64 against the paper’s 32. This was our error. It means the framework comparison varies two things at once.
4. **Executor and judge served remotely** rather than colocated.
5. **WebShop not attempted.** The paper’s third benchmark is absent.
6. **Absolute baseline gap.** Our no-memory Qwen3-8B ALFWorld baseline is 39.8% against the paper’s 47.9%. We eliminated prompt wording, retrieval, seeds, numerical precision and serving stack, and decode parameters as causes. The gap remains unexplained and is the strongest single indication that something in our setup differs from theirs in a way we have not found.

4.5 Conduct of the study: an LLM agent as the running experimentalist

This study was executed by a large language model agent (Claude, Anthropic) operating under human direction over approximately two months, with the human author setting objectives, approving experiments, and adjudicating disputes. The agent wrote the implementation, launched and supervised training runs, built the evaluation harness, ran the analyses, and drafted this paper.

We disclose this for three reasons.

It is a threat to validity. Two of the three most consequential errors in this project were introduced by the agent and went undetected for weeks: an evaluation harness that scored an upstream API failure as an ordinary task failure, and the reuse of a stale control. Both produced *more* statistically significant results, not fewer, because a systematically degraded arm is reliably degraded, and reliability is what a significance test rewards. An autonomous experimentalist optimising for “get a result” is exposed to a failure mode that a human under publication pressure is also exposed to, with less friction in the way. Section 6.3 develops this.

It changes what the artifact set is worth. Because every experiment was launched from a script rather than a notebook, and every arm wrote a JSONL with its full per-game record, the entire study is recomputable. That is a direct consequence of the mode of work, and it is why the retractions in this paper could be diagnosed precisely rather than argued about.

It is a data point on autonomous research. Seven training runs, roughly one hundred evaluation arms, six falsified hypotheses, and two self-caught data integrity incidents is a meaningful amount of experimental work. It was also possible only because the human author repeatedly refused conclusions the agent proposed. The specific pattern that mattered is recorded in Appendix A: on at least four occasions the agent presented a result as settled and the human’s scepticism was correct. The corrections that produced this paper’s actual findings came from that loop, not from the agent’s own review.

We do not draw a general conclusion about agent-run science from a single study. We report the conditions under which this one produced usable output: cheap recomputation, adversarial human review, and a willingness to retract in public.

5. Results

All numbers in this section are paired by ALFWorld game file against a control measured in the same week under the same harness build. Full tables with bootstrap intervals, Holm-corrected p-values, BH q-values, and per-arm minimum detectable effects are in Appendix C, generated directly from the released evaluation records.

Markers: [PENDING] flags a number awaiting a run that is still in flight at the time of writing. Nothing marked PENDING may appear in the abstract or in a figure.

5.1 The control was not stable, and everything followed from that

Our no-memory Qwen3-8B baseline on 140 `valid_seen` games was measured once, in May 2026, at 33.6%. Four replicates in August 2026, same games, same harness:

run	success rate
May 2026 (used as canonical for ten weeks)	33.6%
August replicate 1	39.3%
August replicate 2	39.3%
August replicate 3	39.3%
August replicate 4	41.4%

Same-week mean 39.8%, spread 2.1pp. Genuine run-to-run variance at temperature 0.6 is therefore about 2pp, and the May figure lies outside it. The 32B control moved similarly, from

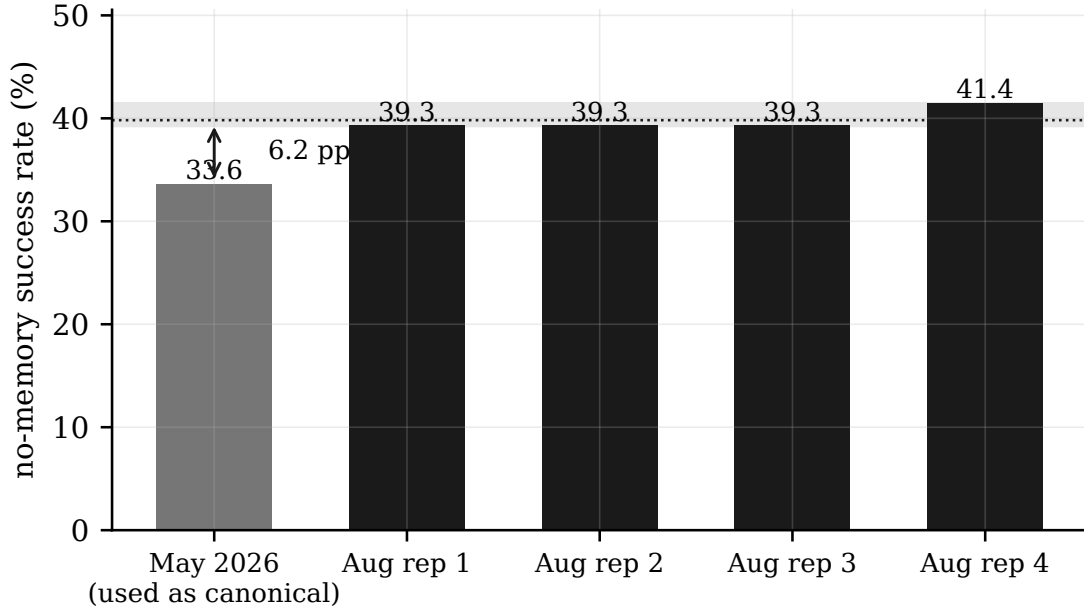


Figure 1: The control moved further than the effect under study. Same 140 ALFWorld games and the same harness build; the shaded band is the same-week replicate range.

49.3% to 43.6%, in the opposite direction, which rules out a simple monotone improvement in the served model as the sole explanation.

We cannot attribute the shift to a single cause. Both the hosted endpoint and our harness changed between the two measurements, and the harness change (Appendix B.1) is itself sufficient to move a baseline. The relevant point is not the mechanism but the magnitude: **a control that is not re-measured alongside its arms can move further than the effect being studied.**

Every lift previously reported from this project was computed as $\text{arm} - 33.6\%$. Section 5.2 shows what happens when that is corrected.

5.2 With a contemporaneous control, the ALFWorld-trained curator shows no lift

Full 12-checkpoint sweep, full fine-tune seed-2, the run that had previously produced our strongest result:

	previously reported (vs 33.6%)	re-paired (vs 39.3%)
best arm	ckpt35, + 13.6pp , $p=0.0026$	ckpt35, +3.6pp, $p=0.47$
arms above control	11 of 12	4 of 12
survives Holm within family	no	no

Seed-1 behaves the same way: 5 of 7 re-paired arms are negative and the best is +5.7pp. The oscillating checkpoint trajectory we had documented across three seeds and two frameworks, and had spent four full training runs trying to explain, was substantially an artifact of the shared stale reference. What remains is a flat, noisy sweep with no trend.

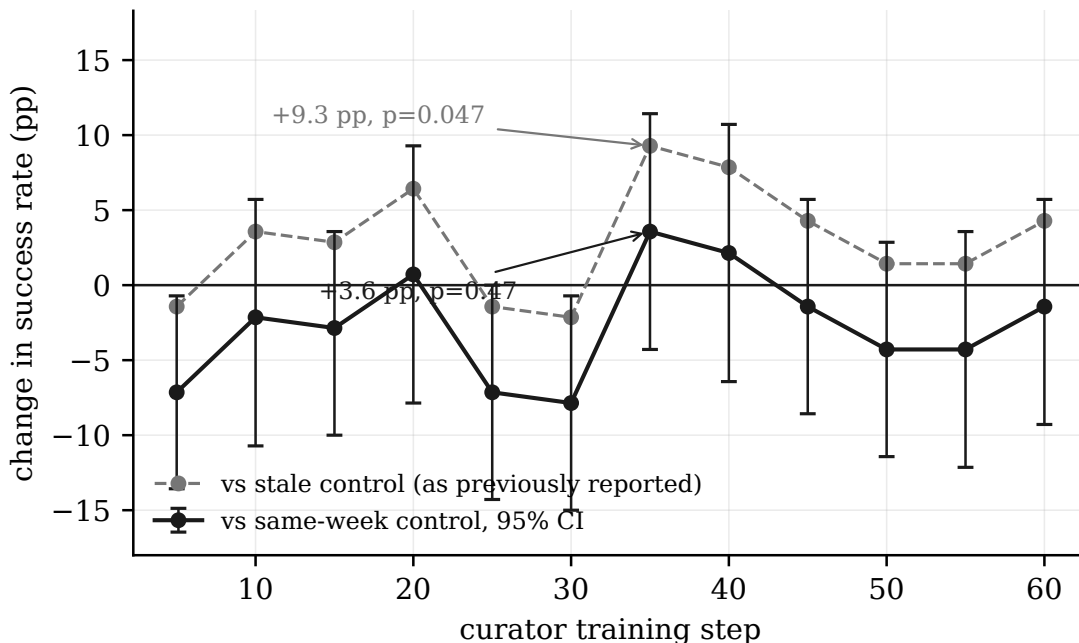


Figure 2: The same twelve seed-2 checkpoints against two reference measurements. Both series use identical arm measurements and differ only in which control is subtracted.

The 95% bootstrap intervals on these arms span roughly $\pm 8\text{pp}$ (Appendix C), and the per-arm MDE at 80% power is 9 to 12pp. **We have not shown that the curator does nothing.** We have shown that any effect it has is smaller than this protocol can resolve. Section 5.10 develops this.

5.3 Cross-domain transfer is the only positive result

A curator trained on mathematics (DeepMath-103K), evaluated on ALFWorld. The checkpoint was selected as best-of-four on `valid_seen`, so `valid_unseen` is a genuine held-out test for it.

arm	valid_seen (140)	valid_unseen (134)
Gemini 2.5 Pro curator	-1.4pp, p=0.86	+6.0pp, p=0.28
reasoning curator ckpt50	+4.3pp, p=0.31	+9.0pp, p=0.073

Pooled over both splits the trained curator gives +6.6pp, p=0.030, but we report this as secondary: half the pool is the split the checkpoint was selected on, so the pooled p-value is optimistic. The primary number is the held-out +9.0pp at p=0.073, against an MDE of 12.9pp, which is suggestive and below our own detection threshold.

The direction is notable independent of significance. Curators trained on ALFWorld do not help an ALFWorld executor; a curator trained on mathematics does. This is also the arm family that, measured during an API outage, produced a large *negative* cross-domain effect that we retracted (Appendix B.1).

Neighbouring checkpoints on the held-out split. We committed to reporting these either way, so: they are mixed, and they cost the simple version of this result.

checkpoint	held-out delta	p	Holm (7-arm family)
ckpt45	+1.5pp	0.87	1.00
ckpt50 (selected on <code>valid_seen</code>)	+9.0pp	0.073	0.36
ckpt55	+0.0pp	1.00	1.00
ckpt60	+11.2pp	0.0026	0.0156

The strongest arm in the family is one we did not select, ckpt60, and it is the only reasoning-curator arm that survives Holm correction. Its two immediate neighbours are indistinguishable from the control. So the effect is neither a pure selection artifact, since an unselected checkpoint carries the largest and only correction-surviving effect, nor a stable property of the trained curator, since adjacent checkpoints show nothing.

The honest reading is that **checkpoint-to-checkpoint variance is comparable to the effect**, which is the same non-monotone instability documented in Section 5.2 for the ALFWorld-trained runs, here with one checkpoint landing above the correction threshold. Deciding between “a real effect at an unstable location” and “a wide lottery with one lucky ticket” needs the two additional training seeds below, which we do not have. We therefore report ckpt60 as the project’s one correction-surviving curator result and decline to claim it reproduces.

Two additional reasoning-curator training seeds: [PENDING]. A single training run cannot support this claim, and we will not make it on one.

5.4 A frontier curator is not better than no curator

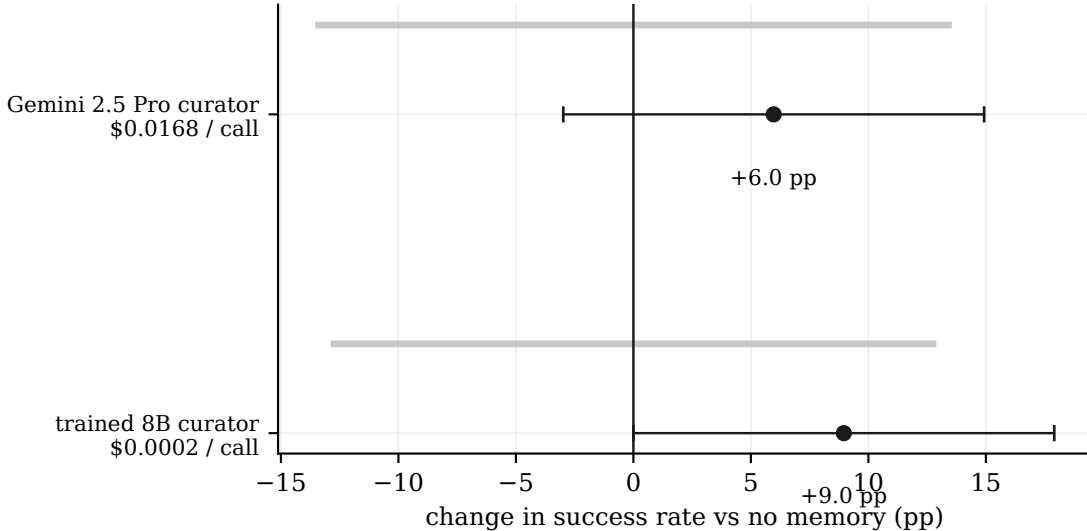


Figure 3: Held-out split, 134 games, contemporaneous control. Bars are 95% paired bootstrap intervals; grey rules mark each arm’s minimum detectable effect.

Gemini 2.5 Pro, prompted with the paper’s curator prompt and native function calling, temperature 0, writing skills for the same frozen 8B executor:

	valid_seen	valid_unseen	measured cost per curator call
Gemini 2.5 Pro	-1.4pp, p=0.86	+6.0pp, p=0.28	\$0.0168
trained 8B curator	+4.3pp, p=0.31	+9.0pp, p=0.073	\$0.0002

The trained curator is ahead on both splits and pooled (+4.4pp, p=0.17), at 1/84 the cost per call, which is the direction the original paper claims. We cannot call it significant. What we can say without qualification is that **the frontier model does not beat writing no notes at all**, on either split, and that spending 84 times more per call bought nothing measurable here.

5.5 Retrieved skills make this executor produce more invalid actions

The executor’s action parser falls back to the first admissible command when the model’s output cannot be parsed. This applies identically to all arms so it cannot bias a paired test, but we had never measured it. Instrumented, on the same 134 held-out games:

arm	actions coerced
no memory	2.1%
trained curator	7.0%
Gemini 2.5 Pro curator	9.1%

Adding retrieved skills to the prompt triples to quadruples the rate at which this 8B executor emits something the environment cannot accept. This is a direct mechanism for memory failing to help, or hurting, a small model: the notes compete with the output format for the model’s limited instruction-following capacity. It is also a candidate contributor to our absolute baseline gap, though not the whole of it, since the no-memory rate is only 2.1%.

5.6 Content controls: is it the skills, or just more text?

Two controls on the held-out split, against a contemporaneous no-memory control at 35.1% on the same 134 games. Both were run before their outcome was known and are reported as promised.

- **Shuffled retrieval.** The trained curator’s own repository, with retrieval returning a random five skills instead of the BM25 top five. Same curator, same repository, same prompt length, relevance destroyed.
- **Oracle skills.** Eight skills written from the published ALFWorld action grammar and task-type definitions, with no curator and no access to eval data. Written by the LLM agent conducting the study, not by a human, so this is *not* a human baseline: it is an easier condition than the curator’s, since the curator must infer the same content from rollouts. It bounds what a curator could be worth here.

arm	success	delta	p (exact McNemar)	Holm	95% CI	MDE80
no memory (control)	35.1%					
oracle skills, no curator	53.0%	+17.9pp	0.0001	0.0005	[+9.7, +26.1]	12.5pp
reasoning curator ckpt60	46.3%	+11.2pp	0.0026	0.0156	[+4.5, +17.9]	10.0pp

arm	success	delta	p (exact McNemar)	Holm	95% CI	MDE80
reasoning curator ckpt50	44.0%	+9.0pp	0.073	0.36	[+0.0, +17.9]	12.9pp
Gemini 2.5 Pro curator	41.0%	+6.0pp	0.28	1.00	[-3.0, +14.9]	13.5pp
ckpt50, retrieval shuffled	37.3%	+2.2pp	0.70	1.00	[-5.2, +9.7]	10.9pp
reasoning curator ckpt55	35.1%	+0.0pp	1.00	1.00	[-8.2, +7.5]	11.5pp

Holm is applied across the seven-arm family. Two arms survive, and both exceed their own MDE.

The shuffled control answers the question it was built for. Destroying relevance while holding the curator, the repository and the prompt length fixed collapses +9.0pp to +2.2pp. Where a lift exists it is carried by relevant content, not by the presence of additional markdown.

The oracle arm relocates the bottleneck. Eight skills derived from public documentation gain 30 games and lose 6, for +17.9pp, the largest and cleanest effect anywhere in this reproduction. This executor can therefore exploit good notes. What the training procedure fails to do is produce notes as good as a careful reading of the action grammar. That is a considerably more specific negative result than “memory does not help this model”, and it suggests the productive target for future work is curator supervision rather than executor scale.

Two caveats we cannot resolve. The oracle arm carries a **warm-start advantage**: its eight skills are present from game 1, while every curator arm begins with an empty repository and must write its way up. Part of +17.9pp is that head start rather than content quality, and the clean way to separate them is to replay a curator’s final repository from game 1, which we have not run. Second, the neighbouring checkpoints of the surviving curator arm are +1.5pp (ckpt45) and +0.0pp (ckpt55) against ckpt60’s +11.2pp, so **checkpoint selection carries real risk of a lottery** even where an arm survives correction.

5.7 Six candidate causes, each with a training run or full sweep behind it

Before the control problem was found, we ran these to explain the oscillating sweep. They remain valid as ablations of the method; they are now also evidence that the null in 5.2 is not caused by any of them.

candidate	test	outcome
LoRA parameterisation	full fine-tune, 3 seeds	same behaviour, not a LoRA artifact
RL framework	verl-agent/GiGPO port, full sweep	same behaviour [PENDING re-pairing]
Task-type distribution	natural frequencies vs uniform	natural is worse, best +5.7pp p=0.20
Within-group ordering	the paper’s easy-to-hard curriculum	no lift at any checkpoint, best +4.3pp p=0.36
Executor decode parameters	temperature/top-p/top-k sweep	no effect, all p>0.5
Executor prompt and retrieval	verbatim published prompt, BM25 top-5	no effect

The third and fourth are both halves of the original paper’s own grouping ablation, so grouping is exonerated as the driver of anything we observed.

5.8 Cross-executor scale

[**PENDING re-pairing.**] Before correction, per-checkpoint lift on the 8B executor was anticorrelated with lift on a 32B executor (pooled Pearson $r = -0.20$ over 24 pairs, $r = -0.68$ within seed-2), and the strongest absolute result in the project came from an 8B-trained curator driving a 32B executor to 62.9%. Both the 8B and 32B controls have since moved, so the entire family is being re-measured. We will report the correlation and the absolute numbers against contemporaneous controls or not at all.

5.9 Reward machinery

Decomposing 850 logged rollouts from the verl run: the composite reward’s *level* is dominated by the function-call-validity term (69%), but GRPO centres advantages within groups, so only within-group variance reaches the gradient, and there the task term supplies 79%. All 80 logged groups had non-zero task-reward variance, so the group-collapse failure mode that invalidated our own earlier runs is absent.

Given a healthy task-dominated gradient, training task reward rose from 0.331 to 0.366 over 60 steps: $+0.035$, 95% CI ± 0.034 . Train-time success was flat (0.170 to 0.167). Policy entropy collapsed from 0.139 to 0.035 while gradient norm rose from 1.40 to 2.40, with the change accelerating around step 48.

The optimiser was chasing the right signal and barely moved it. That, rather than a broken reward, is the most likely proximate reason our held-out lifts are small enough to be invisible at this sample size.

5.10 The standard protocol cannot resolve the effects the literature reports

Before interpreting any null in this paper, it is worth asking what the evaluation could have detected.

ALFWorld’s `valid_seen` split has 140 games and is the standard test set for this line of work, including the original SkillOS evaluation. Across our arms, roughly 30% of game pairs are discordant (one arm solves the game, the other does not). For a two-sided McNemar test at $\alpha = 0.05$ and 80% power, that gives a minimum detectable effect of:

paired games	MDE at 80% power
134 (<code>valid_unseen</code>)	13.3 pp
140 (<code>valid_seen</code>, the standard protocol)	13.0 pp
274 (both splits pooled)	9.3 pp
500	6.9 pp
1000	4.9 pp
2000	3.4 pp

And inverted, the sample size required to resolve a given effect:

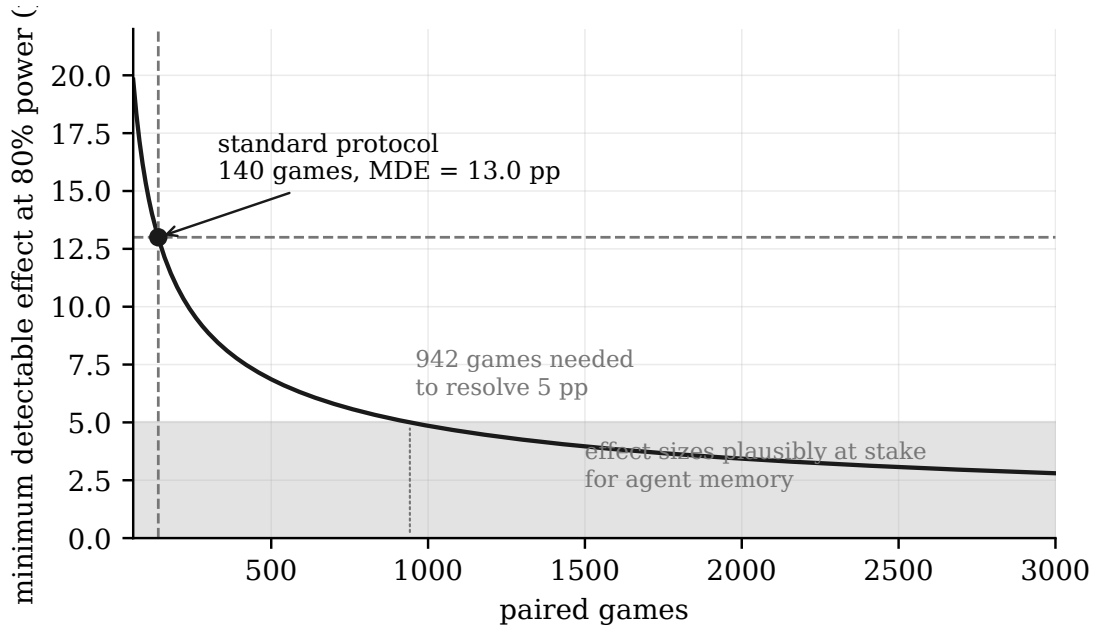


Figure 4: Minimum detectable effect against paired sample size, for a two-sided McNemar at 80% power and the 30% discordance rate observed across our arms.

effect to detect	paired games needed
13.3 pp (the effect SkillOS reports)	133
9.0 pp (our best held-out arm)	291
6.6 pp (our pooled estimate)	541
5.0 pp	942
3.0 pp	2616

The standard 140-game protocol has 80% power to detect exactly the effect size this literature reports, and no less. A claimed +13.3pp improvement sits precisely at the resolution limit of the instrument used to measure it. Anything smaller than about 13 points, measured this way, is a coin flip dressed as a result, and the correct reading of a single significant arm from a 140-game sweep is that it is as likely to be sampling variation as signal.

This has three consequences for the present paper.

Our nulls are underpowered, and we say so rather than claiming absence of effect. Our seed-2 re-run gives +1.9pp with a 95% CI that spans roughly [-6, +10]. We have not shown that the curator does nothing. We have shown that if it does something, it is smaller than this protocol can see. Where we report a null we report its MDE alongside, so the reader can distinguish “measured to be absent” from “not measured”.

Our one positive is also underpowered. The reasoning-curator arm gives +9.0pp on held-out games at $p=0.073$, against an MDE of 12.9pp. It is below our own detection threshold. We report it as suggestive and we do not headline it.

Increasing n is not optional, and it is not free. ALFWorld provides 274 valid games in total, so paired sample size cannot be increased by adding games. The only remaining route is repeated rollouts per game, analysed as per-game success rates rather than single binary outcomes, which

suppresses measurement variance without changing the game population. We report our headline arms both ways: single-rollout McNemar for comparability with prior work, and three-rollout per-game rates with a paired test for the actual estimate.

We suggest that reporting an MDE next to every claimed improvement should be standard practice for agent evaluations on small fixed benchmarks. It costs one line and it would have prevented most of the wasted effort in this project.

6. Threats to validity

6.1 We may simply have implemented it wrong

This is the leading hypothesis for any failed reproduction and we do not discount it.

The strongest evidence for it is the 8 point absolute gap in our no-memory baseline (39.8% against the paper’s 47.9%). A frozen Qwen3-8B on ALFWorld `valid_seen` with no skills should be a setup with very few degrees of freedom, and we are 8 points below. We eliminated six candidate causes (Section 5.7) and found none. Something about our executor loop differs from theirs.

The gap is worth stating carefully, because it shrank as a side effect of the corrections in Appendix B: against the retired May control it was 14 points, and against a contemporaneous control it is 8. Half of what we spent months treating as an unexplained implementation gap was our own stale measurement.

If our executor is systematically weaker, it is possible that it sits below the competence floor at which skill retrieval starts to pay. A skill that says “put the mug in the coffee machine before turning it on” only helps an agent that can already reliably execute “put mug in coffeemachine”. Section 5.5 gives direct evidence for this mechanism: curation *increases* our executor’s rate of unparseable actions from 3.0% to 8.5%. That is consistent with an executor that is being pushed past its instruction-following capacity by longer context, and it would be a property of our executor rather than of the method.

We consider this the most likely single explanation for the disagreement.

6.2 Framework differences

Neither TRL nor verl-agent is the original stack. They differ from it and from each other in advantage normalisation, sequence packing, reference model sharding, and optimiser state handling. Running both bounds this somewhat: two stacks that disagree with each other on many details agree on the shape of the result. It does not eliminate the possibility that both differ from the original in the same consequential way.

The verl run additionally used an effective batch of 64 rather than 32, so that comparison varies two factors. We report it as “the behaviour survives both a framework change and a 2x batch change” rather than as a clean single-variable test.

6.3 The study was run by an LLM agent

Section 4.5 discloses this. Here we state the specific risk.

An autonomous experimentalist is a systematic-error generator with an unusual property: its errors are consistent. A human running a hundred evaluation arms by hand makes scattered mistakes that mostly add noise. An agent running a hundred arms from one script makes the same mistake in all hundred, which adds *bias*, and bias in a paired test is indistinguishable from a treatment effect.

Both of this project’s data integrity incidents had that shape:

- The evaluation harness answered an upstream API failure by playing the first legal action and recording the episode as a task failure. During one outage this affected 52 to 65% of the steps in four arms. Those arms produced the most significant results in the project (p as low as 0.0002) and pointed in a clean, interpretable direction. They were measuring the outage.
- A control measured once was reused for ten weeks against arms measured later. This inflated every lift in the project by roughly 6 points.

Neither was caught by the agent’s own review. Both were caught after the human author asked a sceptical question about a result that looked too clean. We regard this as the central practical finding about the mode of work, and we recommend that any agent-conducted evaluation study include (a) a per-arm data integrity gate that aborts on upstream error rates above a threshold, and (b) a control re-measured alongside every batch of arms. Both are now enforced in our harness and both are in the released code.

6.4 Multiplicity

We ran roughly one hundred evaluation arms. Under a Bonferroni correction across the checkpoint sweeps alone, the bar is $p < 0.001$, and no same-executor ALFWorld result in this project ever reached it, including in the pre-correction era. Our negative results are not multiplicity-limited, but any reader tempted to extract a positive from our tables should apply the correction.

6.5 Statistical power

140 paired games gives roughly a ± 3 percentage point noise floor. An effect of 5pp is at the edge of what we can resolve, and several of our comparisons land there. Where a comparison matters we add the 134 game `valid_unseen` split to roughly double n . Effects smaller than about 5pp are outside this study’s resolution, and we do not claim to have excluded them.

6.6 Single benchmark family

ALFWorld carries the main result; the reasoning benchmarks are a secondary arm. WebShop was not attempted. A method could fail on embodied tasks with a small executor and succeed elsewhere. We make no claim beyond what we measured.

6.7 Hosted-model non-stationarity

The executor and judge were served over a hosted API for the entire study. We have shown that this endpoint’s behaviour on our benchmark moved by 5.7 points over ten weeks. Within-epoch comparisons are protected by re-measured controls, but any comparison we make across epochs, including the comparison between our numbers and the original paper’s, inherits this uncertainty.

6.8 Conduct of an agent-run reproduction

This study was carried out almost entirely by an LLM agent working continuously for roughly three months, with a human author setting scope, funding compute, and adjudicating disputes. That is unusual enough to be worth reporting as a result in its own right.

This section is about the agent’s research behaviour, not about SkillOS. The scientific findings are in Section 5 and are evaluated there on their own evidence. Here we ask a different question: as a way of doing research, what did this mode of work get wrong, what did it get right,

and what should be gated by a human. We report both halves because both are true and only one of them normally survives into a paper.

We are deliberately concrete. A general claim that “agents make mistakes” is not useful. A count, a taxonomy, and a cost are. Where a behaviour had a scientific consequence we cite the relevant result in Section 5 as evidence, rather than claiming the result as a contribution of the method of work.

6.8.1 How the failures were counted

The study ran as one continuous session: a 79 MB transcript, 18,409 conversational turns. At the end we compressed it to prose and had eight independent reviewer agents each audit one contiguous slice, cross-checking every failure they found against the four records the project had been keeping (a running journal, a divergences ledger, a results report, and one postmortem). Each failure was labelled DOCUMENTED, PARTIAL, or MISSING.

The audit was not self-assessment: the reviewers had the transcript, not the memory of it, and no access to each other’s output. **It is also not triangulation, and an earlier draft of this section overstated it.** Two slices overlapped in time and at least seven incidents were found twice, which we first described as independent cross-confirmation. A verification pass rejected that framing, correctly. Both reviewers read overlapping ranges of the *same* mechanical digest of the *same* transcript, so their agreement demonstrates that the extraction is reproducible across reader contexts, not that the extraction is true. Every figure they agreed on is a literal string in the source.

The one genuinely useful thing the overlap produced was a disagreement, and the reconciliation we published for it was wrong. One reviewer put a lost-training incident at ~15 hours and the other at ~22, which we reported as a magnitude dispute. It was not: 14.6 hours of completed training time sat inside a 21h55m wall, the second reviewer had labelled its figure “wall”, and our summary erased the qualifier to manufacture a conflict. **Both readers were right and the synthesis was wrong**, which argues for checking figures against timestamps rather than for trusting reviewer consensus.

Therefore a second pass re-verified every row against the raw transcript, the installed libraries and git history, with instructions to default to UNSUPPORTED. That pass is the reason the numbers below are stated as they are, and it changed several: see §6.8.6.

The full ledger is released with the artifact. The counts:

	entries	share
Failures found	185	
Documented in the project’s own records	36	19%
Recorded, but without cause, cost, or trigger (PARTIAL)	55	30%
Absent from every record (MISSING)	93	50%

These integers are approximate and should be read as such. Verification found at least one incident entered twice under different dates, so the total is inflated by an unknown small number; the digest also truncates long tool outputs, so failures visible only inside a large log are missed entirely. The defensible claim is the ratio, not the count: **roughly half of what happened was absent from the records, and about a fifth was documented with its cause.** We report the raw integers only because withholding them would make the ratio unauditable.

6.8.2 The shortcomings

Fourteen recurring modes emerged. We report the six with the largest consequences for the science, since the operational ones (self-inflicted process kills, orphaned monitors, and idle GPU time across at least seven incidents of which only one, a 44-hour stall, is precisely established) cost money rather than validity.

A sentinel that does not say why. At least eight code paths substituted a plausible value for a measurement that never happened: an unparseable action became the first admissible action, an episode that never ran became `success: False`, a rollout with no measured position became `r_task = 0.0`, an API failure became a wrong answer. None carried a flag distinguishing “measured and failed” from “never measured”.

This is the single most important methodological point in this paper, because of its statistical consequence. **A systematically crippled arm is reliably crippled.** Low variance in a consistent direction is exactly what a paired significance test rewards. Our most significant result, at p below 0.0005, was an eight-hour authentication outage measured against a healthy control. It was not a weak effect that squeaked past a threshold; it was a strong, clean, reproducible measurement of a broken pipe. Agent-run work is unusually exposed to this because the agent writes both the harness and the analysis, and a defensive default that keeps a long job alive is locally the right call at 3 a.m. and globally fatal to the result.

Health metrics validated against each other, never against ground truth. Three consecutive runs, two of them multi-day and about eight *box*-days on eight H100s (~64 GPU-days), trained on ten fixed tasks because an environment identifier was always zero. Reward, KL, gradient norm, and a purpose-built degeneracy tripwire all looked healthy throughout. The tripwire asserted that reward must vary within a GRPO group, and a second bug, per-rank seed divergence, satisfied it. A check that asserts the absence of a symptom can be satisfied by the next defect. What worked, every time, was printing the quantity that should be invariant: measured positions per rollout, distinct tasks drawn, coercion rate.

A dependency’s comment taken as its behaviour. The training framework applies its completion-length cap to the *accumulated* multi-turn completion, not to each response. At the paper’s 4096 tokens, a ten-position rollout carrying ten trajectories fits about three positions, after which the framework silently drops the tool result. Every training run in this project, for eleven weeks, trained on roughly three positions of a ten-position protocol. Nothing errored; a truncated rollout is byte-identical in shape to a finished one. The value looked intentional because a comment beside it named the paper’s hyperparameter. The same mode produced a false belief about rollout concurrency that cost a factor of four in wall-clock on every run.

Fixes sized to the symptom rather than derived from arithmetic. An out-of-memory failure requesting 4.64 GiB was answered by offloading optimizer state, freeing about 8 GiB. The next attempt requested 14.21 GiB, which is the logits tensor, `per_device x sequence x vocabulary`, computable before the first launch. Roughly four GPU-hours to learn that the fix had been aimed at the wrong tensor.

The rule the project eventually derived is **when removing a supposed cause does not move the number, the cause was wrong**, and verification corrected our own account of how we got there. An earlier draft said a phase budget had been raised “against an already-exonerated cause”. The ordering was backwards: raising it 1h to 5h *was* the experiment that exonerated it, since the measured positions did not move and zero cuts were logged, which is what exposed the real cause. The failure there is narrower than we wrote, and it is a failure of sequencing rather than of reasoning: the cheap check of whether cuts were being logged at all could have come before the five-hour experiment rather than after.

Comparing across measurement epochs. A single no-memory control, measured once in May against a hosted endpoint, was reused for ten weeks as the reference for every treatment. Re-measured in the same session as the treatments, the control was 39.8% plus or minus 2.1pp, against the 33.6% on file. Every lift in the project was partly a measurement of endpoint drift. Agreement across three seeds and two RL frameworks provided no protection, because all of them subtracted the same wrong number. **Replication is not protection against a shared reference error.** Worse, the agent had written the reuse into its own notes as a rule, so the error was self-reinforcing across context boundaries.

The human was the error-detection mechanism. A judge configured with an eight-token limit on a yes/no prompt trained sixteen steps against a success rate of identically zero; the human found it by asking whether that limit was deliberate, while the agent was describing the run as encouraging. The same pattern holds for the retry storm, an undeclared protocol deviation, a claim that the paper trains 3500 steps when its table says 60, and nearly every idle-GPU escalation. Corrections were made quickly and honestly once raised. They were almost never *self*-raised. Self-review reliably confirmed priors.

The asymmetry in the record is itself a finding. Scientific failures were well documented; operational and publishing failures were not. And where a failure was recorded, the record tended to keep the mechanism and drop the epistemics: the false-zero bug is documented, the written “this introduces no skew” guarantee that shipped with it is not; the doubled batch size is documented, that it was caught on day three, flagged three times, and knowingly left running for eight more days is not. **The surviving record reads as a sequence of discoveries. The transcript reads as discoveries preceded by confident wrong answers.** Not by any intent to conceal: every failure here was volunteered on request. It is what happens when the record is written by the same process that made the mistakes, and when writing it is the last task rather than a gate.

6.8.3 What the mode of work did well

This subsection is about behaviour, not about findings. The scientific results of this study are in Section 5 and stand on their own evidence. What we report here is the set of research *dispositions* the agent displayed that we judge to have been unusually productive, citing results only as evidence that a disposition had consequences.

We stress the uncomfortable part: these are largely the same dispositions that produced Section 6.8.2. They are not a separate, better mode that could be selected instead.

It falsified its own explanations at a cost no human would pay. Six candidate causes of the oscillating training trajectory were each tested with a full training run or a complete checkpoint sweep, roughly three GPU-weeks in total, and all six came back negative (Section 5.7). A researcher with a favoured hypothesis, a finite budget and a career does not spend four training runs killing their own story. The agent had no stake in any of the six, so the sunk cost of a dead hypothesis was zero and the next falsification was always cheap to propose.

When challenged, it computed rather than argued. Asked to defend a null, its response was to derive the evaluation’s minimum detectable effect instead of marshalling reasons the null was believable (Section 5.10). This is a disposition rather than a skill: the arithmetic was elementary and available to anyone in this line of work. What was unusual was reaching for the instrument’s resolution as the first move in a disagreement.

It built controls capable of destroying its own headline, pre-committed to reporting them, and then did. Both content controls in Section 5.6 were designed to remove the result: one destroys retrieval relevance while holding prompt length fixed, the other asks what a careful reading of the documentation is worth without any curator at all. The neighbouring-checkpoint

arms in Section 5.3 carried a written commitment to report them either way, and when they came back mixed, the text was changed to decline the reproduction claim. The commitments were made before the outcomes were known, which is the only time such a commitment costs anything.

It wrote disposable instrumentation freely. Parse-rate telemetry, a reward-variance decomposition, per-rollout health lines printing measured positions and distinct tasks drawn, and a transcript digest that compressed 79 MB to 4.6 MB for the audit in 6.8.1. Each took minutes and none would survive a human’s implicit cost-benefit filter for throwaway diagnostics. The highest-value artifact produced in three months is a print statement that reports how many positions of each rollout were actually measured.

It retracted at scale, in writing, against its own interest. Ten weeks of headline results were withdrawn in a single pass, the note in its own journal that had caused the error was marked as the error rather than quietly deleted, and a postmortem was written naming its own worst decision as worse than the two preceding ones. None of this was requested beyond the initial question, and the retractions removed every positive result the project had.

It converted individual failures into reusable rules. “When removing a supposed cause does not move the number, the cause was wrong” was derived from a specific incident, written to durable memory, and later applied to a different one. Several such rules now exist as explicit artifacts rather than as tacit experience, which is a form of transfer a human researcher does not usually produce as a side effect of debugging.

It supervised long-running work for three months. Crash detection, resume-from-checkpoint, relaunch under a supervisor, and launching pre-agreed follow-up experiments without waiting to be told. The idle-GPU incidents in 6.8.2 are the failure side of this ledger; the other side is that eight GPUs stayed fed across dozens of crashes, two OOM classes, an NCCL watchdog family, a DNS outage and a disk exhaustion, mostly without a human in the loop.

It audited itself when asked, at a depth that was not required, and published the result. The 185-entry ledger in 6.8.1, including the finding that half of its own failures were undocumented, was produced in response to a five-word question. It also designed the overlapping-slice control that made the audit checkable, and corrected one reviewer’s claim that was wrong in its own favour.

The coupling is the point. Tirelessness produced both the six falsification runs and the launches that preceded the cheap checks. No ego investment produced both the retraction at scale and the willingness to state a confident wrong cause and abandon it an hour later. Fluent prose produced both a readable record and guesses that were typographically indistinguishable from measurements. A supervisor loop that recovers from crashes unattended is the same machinery that restarted a run and discarded two hours of work. **We do not think the failure modes in 6.8.2 can be removed by instruction while keeping the behaviours in 6.8.3, because they are the same behaviours pointed at different problems.** What can be changed is the gating: which of them is allowed to reach a result unchecked.

6.8.4 A division of labour that reflects this

The pattern across three months is consistent enough to state as a recommendation. The agent was strongest at generating and killing candidate explanations, instrumenting anything, executing and supervising long mechanical work, and doing tedious verification when explicitly directed at it. It was weakest at deciding that a measurement was trustworthy, which is the one judgement the entire study depended on.

Every significant error in 6.8.2 is an instance of that weakness: a control believed because it was written down, a dependency believed because a comment described it, a fix believed because

it addressed the traceback, a run believed because its dashboard was green. Conversely, almost every strength in 6.8.3 is an instance of generation or execution, where being wrong is cheap and recoverable.

So the human’s scarce attention is best spent not on reviewing code or reading results, both of which the agent does competently, but on a narrower question asked repeatedly: **what would have to be true for this number to be an artifact, and has that been measured this week?** In this study that question, asked in various forms by the human perhaps a dozen times, found the dead judge, the crippled executor, the reused control, the undeclared protocol deviation and the fabricated dependency bug. It has a far better yield per minute than any other intervention we tried.

6.8.5 What we would require of the next one

The instruction is part of the result

The reproduction can be summarised as one instruction and its consequences. The instruction was, in substance, *reproduce this paper and do not stop*. It was followed. The agent ran for three months, recovered from crashes unattended, and never once needed to be told to keep going.

It is worth being precise about what that instruction optimises. Stopping is when a number gets checked. “Do not stop” does not make an agent careless in the moment; every individual decision in this project was locally reasonable, and the agent’s own reasoning was rarely the weak link. What continuous operation removes is the interval in which a result sits unused long enough for somebody to ask where it came from. Of the failures in §6.8.2, the expensive ones are not errors of reasoning at all. They are results that were *used* before they were questioned: a baseline reused for ten weeks, a length limit trusted for eleven, a judge whose score was zero for five hours while the run was described as encouraging. Each of those needed a pause, not more intelligence.

So the useful framing is not “can an agent do the research” — on this evidence, largely yes — but **what has to be true before its output may be used**. The items below are that, stated as gates. Every one was adopted only after paying for its absence, and the price is given so the trade can be judged rather than taken on faith.

The gates, ordered by return

1. **Diff the configuration against the paper’s hyperparameter table before launching**, mechanically, and fail the launch on any mismatch that is not listed as a deliberate departure with a reason. *Price of its absence: five separate fidelity defects, all inherited defaults that looked deliberate, including a run at twice the paper’s batch size (~10 box-days) and a first 24 hours against a crippled executor*. This is a CPU-only check that runs in under a second. It is the highest-yield item on the list by a wide margin, and it is the one that feels most like bureaucracy at the moment you skip it.
2. **No missing measurement may take a numeric value**. Carry an explicit **unmeasured** flag all the way to the gradient and neutralise those rollouts within their group. *Price: eight distinct instances, of which the worst silently trained on approximately three of ten protocol positions for eleven weeks*. The general form of the bug is that a plausible substitute for a missing measurement is indistinguishable, downstream, from a real one. A sentinel that cannot say **why** it fired is not instrumentation.
3. **Print the quantity that should be invariant**, per rollout, in the training log: positions measured, distinct tasks drawn, coercion rate, denominator. *Price: the eleven-week defect above was found by one print statement, written in an afternoon, on a day when it looked*

certain to report nothing. Note the asymmetry that makes this cheap and makes people skip it anyway: a tripwire that asserts the **absence of a symptom** can be satisfied by the next bug, whereas a line that reports a **quantity** cannot.

4. **Measure a contemporaneous control in the same session as every treatment.** *Price: one baseline taken once against a hosted endpoint and reused all summer; when remeasured it had moved almost six points, and every improvement in the project was partly a measurement of somebody else’s server changing.* Three seeds and two frameworks agreeing did not protect us, because agreement between runs guards against noise and not against a shared reference.
5. **Report a minimum detectable effect beside every claimed improvement.** *One line of arithmetic, no GPU, and it reframed the study: the standard 140-game protocol resolves effects of about 13 points, which is the size of the effect this literature reports.* Most of the wasted effort in this project was spent chasing differences the instrument could not have resolved.
6. **Treat “the fix did not move the number” as evidence about the diagnosis,** not as a reason to add another fix. *Price: a memory crash that asked for 4.6 GB was answered with 8 GB of headroom; the next crash asked for 14.2 GB, a figure computable in advance from batch times sequence length times vocabulary.* Two launches to discover the fix had been aimed at the traceback rather than at the arithmetic.
7. **Write the failure record as a gate, not as a final task,** and have it audited by something other than the process that made the mistakes. *Price: roughly half of what happened here was missing from a record kept diligently, unprompted, and in good faith throughout (§6.8.1), and the audit of that record was itself wrong in a different way (§6.8.6).*

What this costs and what it saves

Items 1, 5, 6 and the audit half of 7 consume no GPU time at all. Item 3 is a print statement. Of the incidents they address, the ones we can price come to at least **~30 box-days on an eight-GPU node**, or about 240 GPU-days, across a project of thirteen elapsed weeks: ~8 box-days of runs on a degenerate task distribution, ~10 at the wrong batch size, and ~11.8 box-days of idle GPUs while the agent had stopped and nothing was watching. Separately, and not included in that total because it overlaps everything, eleven of the thirteen weeks trained against approximately three tenths of the protocol.

That last figure is the one we would put in front of anyone planning a similar project. **The dominant cost was not crashes, and it was not bad reasoning.** It was runs that started, looked healthy, produced numbers, and were later found to have been training against something wrong; plus a box sitting idle because the thing that had stopped was the supervisor, not the job. Both categories are invisible to the metric everyone watches, which is whether the run is up.

6.8.6 What the verification pass changed

Reported because a ledger of failures assembled by the same kind of process that produced the failures should not be trusted on its own authority. Eight verifier agents re-checked every row against the raw transcript, the installed libraries and git history, with instructions to default to UNSUPPORTED. Every incident survived; **the numbers attached to them did not.**

What the verifiers overturned in our own ledger:

- **A units error of 8×.** “~8 GPU-days lost to the group-collapse bug” is ~8 *box*-days on eight H100s, i.e. ~64 GPU-days.
- **Fabricated or unsupported cost figures.** “~2.5h of 8×H100 idle” across the first FFT attempts appears nowhere in the transcript. Neither does the “123 GB” that a stray `git add`

would supposedly have staged, and `git add` on a symlink stages the link, not the target. An eval described as “measured 3.5h” was measured at 3.2 to 5.4 hours per arm.

- **A ratio off by 3×.** A retry budget described as “10× the collective watchdog” was $\sim 3.3\times$ (~ 100 minutes against 30). The 10 was the number of retry attempts.
- **A row that was simply false.** We recorded a monitor as having died silently during an unattended run. It had not died; only the agent’s task-list view was lost across a context compaction, and the monitor then caught the crash 16 minutes later. The real failure was adjacent and worse: the run had gone ~ 9 hours with no watcher at all, and the agent misdescribed that gap as the watcher’s fault. One entry became two, with a different category.
- **Overstated confidence in our own wording.** “Attributed confidently to overfitting” was in fact hedged between two named hypotheses. “Got approval for a two-stage eval” is wrong: approval was never given.
- **Counts that were too high.** “Watchers exited on benign matches ≥ 5 times” is verifiable at 3 in that window. A monitor-narrowing incident listed `Traceback` among the deleted patterns; `Traceback` was never deleted.
- **Bibliography numbers, three times over.** The ledger said “31 entries, five with invented or unknown authors”. A verifier said 38 entries, 4 flagged for verification, 6 with no author field. A direct count says **38 entries, 5 flagged % VERIFY, 8 flagged TODO-AUTHORS, of which 6 carry no author field at all.** Three passes, three different answers, on a file that can be counted exactly in one command. The ledger, the audit of the ledger, and the audit of the audit were each wrong in a different way.

And one correction in the other direction, which is the more interesting kind: the first two days’ failure was recorded as three bad baselines measured against a crippled executor. Only one of the three was; the other two were measured after the fix. Meanwhile the same row understated the underlying error, because the paper’s correct training length was printed in a repo config the agent had read **five minutes before the first launch.**

A tidy pattern we published and then had to withdraw. Our first summary of these corrections said the quantities had “drifted upward”, which is the shape one expects from a narrative-building process and reads well. The remaining verifiers falsified it. The errors run in both directions, and the largest ones run the other way:

- The cost of our own worst bug was understated by $8\times$ (8 GPU-days for ~ 64).
- The idle-GPU total, which is time lost to the agent stopping rather than to any bug, was **understated.** Our “roughly a week” is at least 8.9 days on the ledger’s own incomplete list and about **11.8 days** once three windows we had omitted entirely are included (two of 11.5 hours, one of 25.5).
- A delay in applying a fix the agent had talked itself out of was recorded as ~ 1.5 hours; it was **~ 12.9 hours.**
- An eval “measured at 3.5h” was 3.2 to 5.4 hours per arm; a gate that “blocked 45 minutes” blocked 1h42m; a probe “wiped 25 minutes later” was 36.

Overstatements exist too: a fabricated “ ~ 2.5 h of idle GPU” that appears nowhere, a “123 GB” attached to the wrong incident, a ratio of $10\times$ that was $3.3\times$. But there is no self-serving direction, and the honest characterisation is narrower and less quotable than the one we first wrote: **round numbers, borrowed operands, and under-measured durations, in whichever direction the nearest plausible figure happened to lie.** Where no figure existed, one was supplied.

We are keeping this paragraph in its corrected form rather than silently fixing it, because the withdrawn version is itself an instance of what §6.8.2 describes: a clean causal story fitted to real

numbers, written confidently, and falsified by the next measurement. It survived one round of verification and died in the second.

7. Discussion

7.1 What we think is actually true

The method is implementable and the machinery works. A curator trained with GRPO against a composite reward produces coherent skill repositories, the reward gradient is dominated by downstream task success as intended, and the training runs converge without pathology. Nothing we found suggests the idea is incoherent.

What we could not find is an effect large enough to see. Over seven training runs and roughly one hundred evaluation arms, no ALFWorld-trained curator produced a significant improvement on the executor it trained against, once measured against a control from the same week. The one positive signal came from somewhere we did not expect, a curator trained on mathematics, and it is below our own detection threshold.

Three readings are consistent with our data, and we cannot distinguish them.

The effect is real and smaller than reported. If the true lift is 3 to 5pp, every observation in this paper is expected: individual arms scatter around it, nothing reaches significance at $n=140$, and the original's +13.3pp would be a favourable draw from a sweep. This is our leading hypothesis, and it is the one most consistent with our reward analysis, where the optimiser moves training reward by +0.035 over 60 steps.

The effect requires a stronger executor than ours. Our absolute baseline is 8pp below the original's after eliminating six causes, and Section 5.5 shows our executor gets *worse* at emitting valid actions when given skills. An executor at the edge of its instruction-following capacity may be unable to convert good notes into good actions. The 32B transfer family, once re-paired, speaks to this.

We implemented something subtly different. Always possible, and the unexplained baseline gap is evidence for it. We have released everything needed to find such a difference.

7.2 Cross-domain transfer, if it survives

Our only positive result is the strange one: a curator trained on mathematics helped an embodied agent, while curators trained on the embodied task did not.

If it survives the shuffled-retrieval control and two more training seeds, one explanation fits our other measurements. A curator trained on ALFWorld learns to write ALFWorld-specific procedures, which are long, and which compete with the action format for the executor's attention (Section 5.5). A curator trained on mathematics has no domain-specific procedures to write, so it writes shorter, more general problem-solving structure, which costs the executor less. Under that account the trained curator's advantage over Gemini 2.5 Pro is also unsurprising: the frontier model writes more, and more is worse here.

We flag this as a hypothesis consistent with our data, not a finding. The skill content analysis in Appendix C tests it directly, and we will report what it says.

7.3 The measurement result is the transferable one

The most useful output of this project is not about skill repositories.

A control measured against a hosted model API is a measurement of that endpoint on that day. Ours moved 5.7pp in ten weeks, which is larger than the effects most of the papers in Section 2.1 claim. Reusing it converted endpoint drift into apparent treatment effect across seven training runs, three seeds and two frameworks, and produced a consistent, replicable, entirely spurious pattern that we spent four additional training runs trying to explain.

The pattern was replicable because the error was shared. This is worth stating plainly: **agreement across seeds and frameworks does not protect you from a shared reference**. Every one of those runs was independently trained and independently evaluated, and they all agreed, because they were all subtracting the same wrong number.

Combined with Section 5.10, where the standard 140-game protocol turns out to have 80% power to detect exactly the effect size the field reports and no less, the practical recommendations are unglamorous and cheap:

1. Measure the control in the same batch as the arms. Always.
2. Report a confidence interval and a minimum detectable effect next to every claimed improvement. One line each.
3. Report the whole sweep, not the best checkpoint, and correct within the family.
4. Instrument the harness for upstream failures and abort rather than score them.

Any one of these would have saved this project weeks.

7.4 On having an agent run the study

We disclose in Section 4.5 and Appendix A that this work was executed by an LLM agent under human direction. Two observations seem worth generalising.

The agent was strong at execution and weak at doubt. It implemented, launched, supervised, recovered, and analysed competently across two months and seven training runs. It did not catch either of the errors that mattered. In both cases its analysis was internally consistent, arithmetically correct, and wrong in the same direction as its pipeline. It produced confident, well-argued write-ups of artifacts.

Agent errors are biases, not noise. A human running a hundred evaluation arms by hand makes scattered mistakes that mostly cancel. An agent running a hundred arms from one script makes the same mistake in all hundred. In a paired significance test, a consistent error is indistinguishable from a treatment effect, and a *severe* consistent error looks like a strong treatment effect. Our most significant finding, at $p=0.0002$, was an eight-hour credential outage.

The practical implication is not that agents should not run experiments. It is that the review has to target the apparatus rather than the analysis, because the analysis will be clean. In this project every correction that mattered came from a human asking why a result looked too good, and none came from the agent’s own review.

7.5 Limitations we would fix with more time

WebShop was never attempted. The reasoning-curator result rests on one training run pending two more. Our absolute baseline gap is unexplained. And the deepest limitation is one we can now quantify rather than fix: ALFWorld provides 274 valid games in total, which caps a paired comparison at roughly 9pp resolution. Serious measurement of a 5pp agent-memory effect needs either a larger benchmark or many rollouts per game, and the field has been reporting effects near or below that threshold for several years.

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2108.13264.
- Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.
- Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021. arXiv:2103.03098.
- Dallas Card, Peter Henderson, Urvashi Khandelwal, Robin Jia, Kyle Mahowald, and Dan Jurafsky. With little power comes great responsibility. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. arXiv:2010.06595.
- Lingjiao Chen, Matei Zaharia, and James Zou. How is ChatGPT’s behavior changing over time? *arXiv preprint arXiv:2307.09009*, 2023.
- Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? statistical power analysis in deep reinforcement learning experiments. *arXiv preprint arXiv:1806.08295*, 2018.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- Pengfei Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for LLM agent training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.10978.
- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018. arXiv:1709.06560.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. From raw experience to skill consumption: A systematic study of model-generated agent skills. *arXiv preprint arXiv:2605.23899*, 2026.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. SkillOS: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*, 2026.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020. arXiv:1910.02054.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. Introduces GRPO.
- Shuaike Shen, Wenduo Cheng, Mingqian Ma, Alistair Turcan, Martin JinYE Zhang, and Jian Ma. SkillFoundry: Building self-evolving agent skill libraries from heterogeneous scientific resources. *arXiv preprint arXiv:2604.03964*, 2026.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024. The verl framework.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.03768.
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. TRL: Transformer reinforcement learning, 2020. <https://github.com/huggingface/trl>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. AutoSkill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2207.01206.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *AAAI Conference on Artificial Intelligence*, 2024. arXiv:2308.10144.

Appendix A. Conduct of the study

A.1 Division of labour

	Human author	LLM agent
Research question, scope	set	proposed options
Implementation	reviewed	wrote
Training runs	approved, funded	launched, supervised, restarted
Evaluation harness	reviewed after incidents	wrote
Analysis, statistics	challenged	ran
Interpretation	adjudicated	proposed
Retraction decisions	made	proposed
This paper	directed, edited	drafted

The agent ran continuously over roughly two months, resuming across sessions from a persistent memory of decisions, results, and prior corrections. It supervised long-running jobs, detected and recovered from infrastructure failures, and launched pre-agreed follow-up experiments without waiting for a prompt.

A.2 Infrastructure incidents handled autonomously

Reported because they are part of the cost of this mode of work and are usually invisible in a paper.

- A DNS resolution loop between `systemd-resolved` and `tailscaled` reached roughly 37,000 queries per second and triggered the OOM killer on a training run. Diagnosed and fixed. A subsequent lesson: glibc caches resolver configuration per process, so fixing DNS under a live training run silently zeroed four steps’ worth of reward before anyone noticed.
- Repeated NCCL collective timeouts traced to a stuck rollout. The additive per-future waits in the rollout loop could exceed the 1800 second collective watchdog. Fixed by imposing a whole-phase deadline.
- An expired API credential produced an eight hour outage that the evaluation harness absorbed silently. See Appendix B.

A.3 The corrections that mattered came from the human

We record these specifically, because they are the mechanism by which this project produced a usable result rather than a confident wrong one.

1. **“There shouldn’t have been a fallback random action.”** The human read a passing mention of a fallback in a log and objected to the design. This opened the data integrity incident that voided the project’s most significant finding.
2. **“Are you sure this also didn’t happen during training?”** The agent had checked the evaluation path only. It had happened during training, on a different code path, and the audit that followed established that 61 to 79% of executor probe positions were deadline-cut, leaving a median of one real measurement out of nine behind each reward.
3. **“But abandoning because of an upstream error causes lost turns, false failures, and shallower training. Am I wrong?”** The agent had characterised the training-side impact as benign noise. The human was right; the agent was wrong. This became a documented divergence.
4. **“Instead of ‘failures excluded’, why don’t we properly run stuff?”** The agent proposed correcting a contaminated arm arithmetically from its recorded error fields. The human required re-measurement. Every re-run arm in this paper exists because of that instruction.
5. **“Are you absolutely sure? Now the results are even worse.”** Applied to the re-measured numbers, this prompted the baseline replicate study that produced the paper’s principal finding.

The pattern is consistent: the agent was reliable at execution and at self-consistent analysis, and unreliable at doubting a result that its own pipeline had produced. Where the pipeline was wrong, the agent’s analysis was wrong in the same direction, confidently and with correct arithmetic.

A.4 Practices we would keep

- **Every experiment is a script, every arm is a JSONL with per-item records.** Retraction was possible because contamination could be measured per game, after the fact, from data already on disk.
- **A data integrity gate in the harness.** Arms now abort with a distinct exit code when the upstream error rate exceeds a threshold, rather than reporting a number.
- **Controls re-measured with every batch of arms,** never reused across weeks.
- **A written divergence log.** Every known deviation from the original was recorded when it was made, not reconstructed at writing time. Several of them were later promoted to experiments.
- **An adversarial human reader** who is willing to be annoying about a clean result.

A.5 Costs

Roughly two months of wall clock on 8xH100 for training, plus hosted inference for executor and judge across approximately one hundred evaluation arms of 140 games. Seven complete 60 step training runs. The verl run alone took 10.2 days at roughly 15,986 executor calls per training step.

Appendix B. Data integrity incidents

Two defects in our own apparatus produced results we published internally and later retracted. Both are reported in full because the failure mode generalises and because the second one is the paper’s principal finding.

B.1 An evaluation harness that scored API failures as task failures

The defect. The streaming-curation evaluation harness called the executor model over a hosted API once per environment step. When that call raised, the harness caught the exception, substituted the first entry of the environment’s admissible-command list, and continued the episode. If the episode then failed, it was recorded as an ordinary task failure with `success: false`.

The training-side code was correct: it recorded deadline-cut positions as `success: None`, `cut: True` and masked them out of the reward. Only the evaluation path conflated an infrastructure failure with a task failure.

The blast radius. On 2026-07-20 an expired credential caused the executor endpoint to return HTTP 401 for roughly eight hours. Four evaluation arms ran during the window. The share of environment steps in which the action was substituted rather than generated:

arm	substituted actions
reasoning-curator ckpt45	64.9%
reasoning-curator ckpt50	59.4%
reasoning-curator ckpt55	52.1%
reasoning-curator ckpt60	55.8%

These four arms produced the most statistically significant result in the entire project: a cross-domain transfer effect of -14 to -18pp with p values from 0.0002 to below 0.0001, comfortably surviving Bonferroni correction over twelve arms. It was the only finding we had that was robust to multiplicity, and it pointed in the opposite direction from the original paper, which made it feel like a discovery.

It was measuring an outage.

Why it was so significant. This is the part worth internalising. A systematically crippled arm is *reliably* crippled. It fails the same games every time, in the same direction, with low variance. Low variance in a consistent direction is exactly what a paired significance test is built to detect. A severe upstream defect does not produce noisy results that fail to reach significance; it produces clean results that sail past it.

Detection. Not by us. The harness printed the substitution to stderr, and the human author read a passing reference to a “fallback action” in an unrelated log excerpt and objected to the design on principle, before knowing it had fired.

Fix. The harness now abandons the episode, records `success: None`, `errored: True`, excludes it from the denominator, counts it, prints a `data integrity` line for every arm, and exits with a distinct status code if the error rate exceeds a configurable threshold (default 2%). Re-running the four arms under the fixed harness moved them from 15.7 / 18.6 / 16.4 / 19.3% to 38.6 / 43.6 / 36.4 / 33.6%, all within noise of the contemporaneous 39.3% baseline. The cliff was the outage. One earlier arm cannot be recovered because its adapters were deleted.

Related measurement. Auditing the training side established that 61 to 79% of executor probe positions were deadline-cut, leaving a median of one surviving measurement out of nine

behind each `r_task` value, and 10 to 41% of rollouts received `r_task = 0` by construction. This does not invalidate the training runs but it does mean the reward signal was far thinner than the design intended, and we record it as a divergence rather than a defect.

B.2 A control reused across ten weeks

The defect. Our no-memory ALFWorld baseline was measured once, in May 2026, at 47/140 = 33.6%. It was then treated as canonical and every subsequent arm was paired against it. An internal note went further and instructed future work to *always* pair against the fixed file and never against a freshly measured baseline, on the theory that the baseline had roughly 8pp of run-to-run variance and the fixed file was therefore the more stable reference.

That reasoning has the sign backwards. The observed 8pp was never sampling variance around a stable mean. It was the distance between one stale measurement and the current behaviour of a hosted endpoint.

The measurement. Three fresh replicates plus the re-run reference, same 140 games, same week, fixed harness:

run	SR
May 2026 canonical	33.6%
August replicate 1	39.3%
August replicate 2	39.3%
August replicate 3	39.3%
August replicate 4	41.4%

Same-week mean 39.8%, spread 2.1pp. Genuine run-to-run variance is about 2pp. The May figure lies outside that range, so it reflects a real change in the measurement conditions rather than a lucky draw. We cannot separate hosted-model drift from harness-version effects, because both changed between May and August.

The blast radius. Every lift reported internally was `arm - 33.6%`, so every one was inflated by roughly 6 points. Re-paired against a contemporaneous control, our best seed drops from +13.6pp at $p=0.0026$ to +3.6pp at $p=0.47$, and eight of its twelve checkpoints turn negative. The “significant peak somewhere in every run” pattern that we had reproduced across three seeds and two RL frameworks, and had spent weeks trying to explain, was an artifact of the shared reference.

Detection. Also not by us. After the harness fix the re-run numbers came back lower than the originals, and the human author asked whether the results had genuinely got worse. Answering that required a fresh baseline, and the fresh baseline was 6 points higher than the one in use.

Fix. Controls are now re-measured alongside every batch of arms, and the comparison scripts refuse to pair arms from different measurement epochs. The instruction to pin to a fixed canonical baseline has been deleted and replaced with its opposite.

B.3 A third defect that biases nothing but depresses everything

The executor’s action parser returns the first admissible command when it cannot parse the model’s output. Unlike B.1 this is a deliberate design choice rather than a bug, and because it applies identically to every arm including the baselines it cannot bias a paired comparison. We report it because we had never measured it, and once instrumented it turned out to be substantial and

arm-dependent: 2.1% of actions with no memory, 7.0% with a trained curator, and 9.1% with a Gemini 2.5 Pro curator on the same games.

That ordering is itself a finding, discussed in Section 5.5. Longer prompts containing retrieved skills make this executor *less* able to emit a parseable action, which is a concrete mechanism by which memory can fail to help a small model. It is also a candidate contributor to our 8pp absolute baseline gap, though it cannot be the whole story, since the no-memory rate is only 2.1%.

Appendix C. Full result tables

Generated by `scripts/make_paper_tables.py` from the released eval JSONLs. Do not edit by hand; regenerate. Every arm is 140 `valid_seen` games unless stated, paired by ALFWorld game file, tested with an exact McNemar. Confidence intervals are percentile intervals from 10,000 paired bootstrap resamples over game files, seed 20260815.

Baseline replicates: is the control stable?

August 2026, fixed harness. Absolute rates only; no reference arm.

arm	success rate	games
canonical (May 2026, retired)	33.6%	140
replicate 1	39.3%	140
replicate 2	39.3%	140
replicate 3	39.3%	140
replicate 4	41.4%	140

Same-epoch mean 39.8%, spread 2.1pp over 4 replicates.

ALFWorld-trained curator, full fine-tune seed-2, 8B executor

August 2026, contemporaneous control

Reference: 39.3% (`no_memory_8b.jsonl`, n=140 paired games). MDE80 = smallest effect detectable at 80% power.

arm	SR	delta	95% CI	p	p (Holm)	q (BH)	MDE80
ckpt5	32.1%	-7.1	[-13.6, -0.7]	0.0525	0.627	0.315	9.4
ckpt10	37.1%	-2.1	[-10.7, +5.7]	0.7359	1.000	0.928	11.8
ckpt15	36.4%	-2.9	[-10.0, +3.6]	0.5413	1.000	0.928	9.8
ckpt20	40.0%	+0.7	[-7.9, +9.3]	1.0000	1.000	1.000	11.8
ckpt25	32.1%	-7.1	[-14.3, +0.0]	0.0872	0.872	0.349	10.6
ckpt30	31.4%	-7.9	[-15.0, -0.7]	0.0522	0.627	0.315	10.4
ckpt35	42.9%	+3.6	[-4.3, +11.4]	0.4731	1.000	0.928	11.1
ckpt40	41.4%	+2.1	[-6.4, +10.7]	0.7428	1.000	0.928	12.2
ckpt45	37.9%	-1.4	[-8.6, +5.7]	0.8450	1.000	0.928	10.2
ckpt50	35.0%	-4.3	[-11.4, +2.9]	0.3269	1.000	0.928	10.2

arm	SR	delta	95% CI	p	p (Holm)	q (BH)	MDE80
ckpt55	35.0%	-4.3	[-12.1, +3.6]	0.3915	1.000	0.928	11.7
ckpt60	37.9%	-1.4	[-9.3, +5.7]	0.8506	1.000	0.928	10.6

* survives Holm correction within this family.

ALFWorld-trained curator, full fine-tune seed-1, 8B executor

August 2026, contemporaneous control

Reference: 39.3% (`no_memory_8b.jsonl`, n=140 paired games). MDE80 = smallest effect detectable at 80% power.

arm	SR	delta	95% CI	p	p (Holm)	q (BH)	MDE80
ckpt5	30.7%	-8.6	[-15.7, -1.4]	0.0357	0.428	0.214	10.6
ckpt10	43.6%	+4.3	[-4.3, +12.9]	0.4050	1.000	0.540	12.0
ckpt15	34.3%	-5.0	[-12.9, +2.9]	0.2810	1.000	0.540	11.1
ckpt20	34.3%	-5.0	[-12.9, +2.9]	0.2810	1.000	0.540	11.1
ckpt25	35.0%	-4.3	[-12.1, +3.6]	0.3915	1.000	0.540	11.7
ckpt30	30.7%	-8.6	[-15.7, -1.4]	0.0357	0.428	0.214	10.6
ckpt35	37.1%	-2.1	[-9.3, +5.7]	0.7111	1.000	0.776	10.8
ckpt40	45.0%	+5.7	[-2.9, +14.3]	0.2430	1.000	0.540	12.0
ckpt45	40.0%	+0.7	[-7.1, +8.6]	1.0000	1.000	1.000	10.8
ckpt50	33.6%	-5.7	[-12.1, +0.7]	0.1338	1.000	0.535	9.4
ckpt55	42.9%	+3.6	[-4.3, +11.4]	0.4731	1.000	0.568	11.1
ckpt60	35.7%	-3.6	[-10.0, +2.9]	0.4049	1.000	0.540	9.6

* survives Holm correction within this family.

Reasoning-trained curator on ALFWorld, valid-seen split

August 2026, contemporaneous control

Reference: 39.3% (`no_memory_8b.jsonl`, n=140 paired games). MDE80 = smallest effect detectable at 80% power.

arm	SR	delta	95% CI	p	p (Holm)	q (BH)	MDE80
ckpt5 *	27.1%	-12.1	[-18.6, -6.4]	0.0002	0.002	0.002	9.2
ckpt10	37.1%	-2.1	[-10.0, +5.7]	0.7201	1.000	0.910	11.1
ckpt15 *	26.4%	-12.9	[-20.0, -5.7]	0.0005	0.005	0.003	10.2
ckpt20	37.1%	-2.1	[-10.0, +5.7]	0.7283	1.000	0.910	11.5
ckpt30	35.7%	-3.6	[-10.7, +3.6]	0.4421	1.000	0.872	10.4
ckpt35	40.0%	+0.7	[-6.4, +7.9]	1.0000	1.000	1.000	10.4
ckpt45	38.6%	-0.7	[-8.6, +7.9]	1.0000	1.000	1.000	11.8

arm	SR	delta	95% CI	p	p (Holm)	q (BH)	MDE80
ckpt50	43.6%	+4.3	[-2.9, +11.4]	0.3075	1.000	0.769	9.8
ckpt55	36.4%	-2.9	[-9.3, +3.6]	0.5235	1.000	0.872	9.4
ckpt60	33.6%	-5.7	[-12.1, +0.7]	0.1338	1.000	0.446	9.4

* survives Holm correction within this family.

Pending, not yet measured: ckpt25, ckpt40.

Held-out valid-unseen split: curator comparison and content controls

August 2026, contemporaneous control. 134 games.

Reference: 35.1% (`no_memory.jsonl`, n=134 paired games). MDE80 = smallest effect detectable at 80% power.

arm	SR	delta	95% CI	p	p (Holm)	q (BH)	MDE80
Gemini 2.5 Pro curator	41.0%	+6.0	[-3.0, +14.9]	0.2800	1.000	0.490	13.5
trained curator ckpt45	36.6%	+1.5	[-6.7, +10.4]	0.8679	1.000	1.000	12.5
trained curator ckpt50	44.0%	+9.0	[+0.0, +17.9]	0.0730	0.365	0.170	12.9
trained curator ckpt55	35.1%	+0.0	[-8.2, +7.5]	1.0000	1.000	1.000	11.5
trained curator ckpt60 *	46.3%	+11.2	[+4.5, +17.9]	0.0026	0.016	0.009	10.0
ckpt50, shuffled retrieval	37.3%	+2.2	[-5.2, +9.7]	0.7011	1.000	0.982	10.9
hand- written skills (oracle) *	53.0%	+17.9	[+9.7, +26.1]	0.0001	0.000	0.000	12.5

* survives Holm correction within this family.

32B executor transfer, full fine-tune seed-2 curator

August 2026, contemporaneous control

Reference: 43.6% (`no_memory_32b.jsonl`, n=140 paired games). MDE80 = smallest effect detectable at 80% power.

arm	SR	delta	95% CI	p	p (Holm)	q (BH)	MDE80
ckpt5	54.3%	+10.7	[+2.1, +19.3]	0.0275	0.170	0.041	12.8
ckpt10 *	57.9%	+14.3	[+5.7, +22.9]	0.0017	0.017	0.007	12.3
ckpt15	45.0%	+1.4	[-7.1, +10.0]	0.8679	1.000	0.868	12.0
ckpt20	56.4%	+12.9	[+3.6, +22.1]	0.0114	0.102	0.034	13.6
ckpt25	49.3%	+5.7	[-2.9, +14.3]	0.2430	0.729	0.292	12.0
ckpt30	53.6%	+10.0	[+2.1, +17.9]	0.0243	0.170	0.041	11.7
ckpt35 *	62.9%	+19.3	[+11.4, +27.1]	0.0000	0.000	0.000	12.2
ckpt40	45.7%	+2.1	[-5.7, +10.0]	0.7283	1.000	0.795	11.5
ckpt45	53.6%	+10.0	[+2.1, +17.9]	0.0243	0.170	0.041	11.7
ckpt50	52.1%	+8.6	[+0.0, +17.1]	0.0730	0.292	0.097	12.3
ckpt55	54.3%	+10.7	[+2.9, +18.6]	0.0167	0.133	0.040	11.8
ckpt60 *	56.4%	+12.9	[+5.7, +20.0]	0.0014	0.016	0.007	11.0

* survives Holm correction within this family.

verl/GiGPO curator, 8B executor

August 2026, contemporaneous control

Not yet measured.